X11 WM

Nathan Warner

Northern Illinois
University

Computer Science
Northern Illinois University
United States

Contents

Introduction

- **The X11 WM role**: An X11 window manager is not the operating system's windowing system. It is a normal X client that is granted special control over the root window's child windows.

  The X server owns the screen, keyboard, mouse, windows, pixmaps, cursors, and other graphical resources. Programs such as terminals, browsers, editors, and your window manager are X clients. They all talk to the X server through the X11 protocol.

  A window manager's job is to manage other clients' top-level windows.

  It usually controls:

  - where top-level windows appear
  - how large they are
  - whether they are mapped or unmapped
  - stacking order
  - focus policy
  - borders/decorations
  - workspaces/tags/desktops
  - fullscreen/maximized/minimized state
  - how user input is interpreted as window-management commands

  It does not normally draw the contents of applications. Firefox draws Firefox's content. Kitty draws Kitty's terminal grid. The window manager controls the container around those windows and their position in the global desktop.

  Your window manager connects to the X server like any other client:

  ```
  Display* dpy = XOpenDisplay(NULL);
  ```

  This gives your program a connection to the running X server.

  The X server stores the actual window objects. In Xlib, a Window is just an integer ID referring to a server-side resource.

  ```
  Window root = DefaultRootWindow(dpy);
  ```

  That root value is not a C object containing the window. It is an ID you send back to the X server when making requests.

  Xlib calls such as **XMoveWindow, XResizeWindow, XMapWindow,** and XDestroyWindow send requests to the server. They do not directly manipulate local objects.

- **X Clients**: An X client is any program connected to the X server.

Each client may create one or more windows. Applications usually create a top-level window, which is the main window the window manager cares about.

A client creates a window using calls such as:

```
XCreateWindow(...)
XCreateSimpleWindow(...)
```

Then it usually maps the window:

```
XMapWindow(dpy, win);
```

Mapping means "make this window visible." But because a window manager exists, the server may not immediately honor that mapping request. Instead, it may redirect the request to the window manager.

- **Top level windows**: A top-level window is usually an application window that is a direct child of the root window. These are the windows the window manager manages.

  Not every window is top-level. Applications often create child windows inside themselves:

  A basic window manager usually ignores internal child windows. It manages the top-level windows that live directly under the root window.

  You can check a windows parent using

  ```
  XQueryTree(...)
  ```

  A window whose parent is the root window is usually a top-level client window.

- **Root window**: The root window is the invisible desktop-sized parent window for the whole screen.

  Every X screen has a root window. It covers the entire display area.

  For a simple one-screen setup:

  ```
  int screen = DefaultScreen(dpy);
  Window root = RootWindow(dpy, screen);
  ```

  The root window is important because clients create their top-level windows as children of it.

The window manager selects events on the root window so it can intercept requests involving the root's child windows.

A minimal window manager usually does something like:

```
XSelectInput(
    dpy,
    root,
    SubstructureRedirectMask | SubstructureNotifyMask
);
```

This is the critical step that makes your program the active window manager.

The window manager is "special" only because it selects SubstructureRedirectMask on the root window.

Only one client may select SubstructureRedirectMask on a given root window at a time.

That means only one window manager can run per screen.

If another window manager is already running, your attempt to select SubstructureRedirectMask fails with a BadAccess error.

Note: The X server creates the root window, not the window manager and not normal applications.

When Xorg starts, it initializes each screen. As part of that initialization, it creates one root window per screen. That root window is the top-level ancestor for all other windows on that screen.

- **SubstructureRedirectMask**: The SubstructureRedirectMask is an Xlib event mask that allows a single client (typically a window manager) to intercept structural requests made to the X server on the subwindows of a parent window

  When this mask is set on a parent window (like the root window), it intercepts requests from other clients to map, configure (resize/move), or circulate child windows. Instead of executing the command, the X server pauses the action and sends the window manager a specific request even

- **SubstructureNotifyMask**: SubstructureNotifyMask is an Xlib event mask used primarily by window managers and system utilities. When selected on a parent window, it alerts the client to any structural or state changes affecting any of its direct child windows.

When the X Server receives SubstructureNotifyMask, it can report the following state-change events for children

  - **CreateNotify:** A new child window was created.
  - **DestroyNotify:** A child window was destroyed.
  - **MapNotify:** A child window was mapped (made visible).
  - **UnmapNotify:** A child window was unmapped (hidden).
  - **ConfigureNotify:** A child window changed size, position, border, or stacking order.
  - **ReparentNotify:** A child window was reparented to a different window.
  - **CirculateNotify:** A child window's stacking order was circulated.
  - **GravityNotify:** A child window was moved due to window gravity.

You should ideally install a temporary X error handler before this so you can detect whether another window manager already owns the root.

1. **normal client:** asks X server to map/configure its window
2. **X server:** sees that WM selected SubstructureRedirectMask on root
3. **X server:** sends event to WM instead of immediately performing request
4. **WM:** decides what to do
5. **WM:** sends XMoveResizeWindow / XMapWindow / XConfigureWindow / etc.

- **Reparenting vs non-reparenting window managers**: A window manager may either manage the application window directly or create a frame window around it.
  - **Non-reparenting WM:** A non-reparenting WM manages the application's top-level window directly.
  - **Reparenting WM**: A reparenting WM creates a frame window, then makes the client window a child of that frame.

```
root window
|__ frame window
    |__ client window
```

    The frame belongs to the window manager. The client belongs to the application. This allows the WM to draw title bars, close buttons, resize handles, shadows, and borders independently of the client window.

- **Override-redirect windows**: Some windows are not supposed to be managed by the window manager. These are called **override-redirect** windows.

  Your WM should usually ignore override-redirect windows.

- **Workspaces**: In a tiling WM, switching workspaces often means:
  - **Windows on current workspace:** Mapped
  - **Windows on hidden workspace:** Unmapped
- **Window lifecycle**: A top-level application window usually goes through this lifecycle:

1. client connects to X server
2. client creates top-level window
3. client sets properties/hints
4. client asks to map the window
5. WM receives MapRequest
6. WM starts managing the window
7. WM positions/resizes it
8. WM maps it
9. user interacts with it
10. window may be resized/moved/focused/unfocused
11. client may unmap it
12. client may destroy it
13. WM removes it from internal state

- **Events to understand**:
  - **MapRequest**: A client wants to show a top-level window.
  - **ConfigureRequest:** A client wants to move/resize/restack a top-level window.
  - **DestroyNotify:** A window was destroyed.
  - **UnmapNotify**: A window was hidden/unmapped.
  - **MapNotify**: A window was mapped.
  - **EnterNotify**: Pointer entered a window. Useful for focus-follows-mouse.
  - **ButtonPress**: Mouse button pressed. Useful for moving/resizing/focusing.
  - **KeyPress**: Key pressed. Useful for WM shortcuts.
  - **ClientMessage**: Client sent a protocol message, often EWMH/ICCCM-related.
  - **PropertyNotify**: A window property changed. Useful for titles, hints, states.
- **Desktop backgrounds**: Setting the background of the desktop usually means setting the background of the root window.

  So, we can convert an image into an X pixmap, set the roots background to that pixmap, and clear / redraw the root window.

Events

Tiling layout model

- **Intro**: A tiling layout model is the part of the window manager that decides: Given a monitor rectangle and a set of managed windows, what rectangle should each window occupy?

  At minimum, a tiling window manager needs to track

```
struct Client {
    Window win;
    int x, y, w, h;

    bool is_floating;
    bool is_fullscreen;
    bool is_hidden;

    int min_w, min_h;
    int max_w, max_h;
    int base_w, base_h;
    int resize_inc_w, resize_inc_h;

    float min_aspect;
    float max_aspect;
};
```

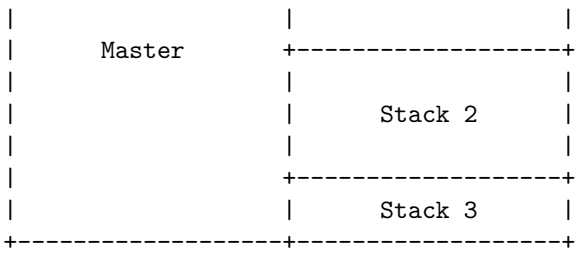
  And something like:

```
struct Workspace {
    Client* clients[MAX_CLIENTS];
    int client_count;

    LayoutKind layout;

    int master_count;
    float master_factor;

    int gap_inner;
    int gap_outer;
    int border_width;
};
```

  The layout algorithm usually processes only tiled clients, floating and fullscreen clients are layered separately.

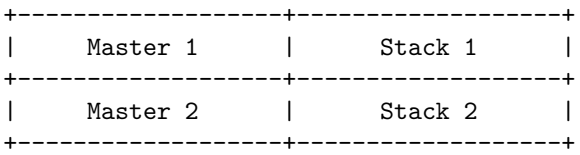
- **Master stack layout**: The screen is divided into two areas:

```
+------------------+------------------+
|                  |                  |
|                  |      Stack 1     |
```

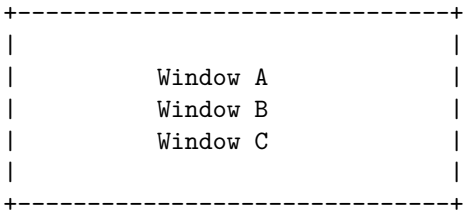


- The focused or first client goes in the master area.
- Remaining clients go in the stack area.
- master_factor controls how much width the master area receives.
- master_count controls how many windows are allowed in the master area.

With multiple master windows, the master area itself is split vertically.



- **Monocle / fullscreen layout**: Monocle means every tiled window receives the full usable monitor rectangle.



They are all the same size, but only the topmost/focused one is visible.

- **Monocle**: The WM still manages the window normally.

  You may keep borders, gaps, and panels visible depending on your design.
- **Fullscreen**: Fullscreen is usually a special state.

  A fullscreen client should typically:
  cover the entire monitor, ignore gaps, ignore borders or use border width 0, appear above normal tiled clients, possibly cover bars/panels, be excluded from normal tiling.

  In EWMH-aware WMs, fullscreen often corresponds to:

```
0  _NET_WM_STATE_FULLSCREEN
```

- **Grid layout**: Grid layout places windows in rows and columns.

```
+----------+----------+----------+
|    A     |    B     |    C     |
+----------+----------+----------+
|    D     |    E     |    F     |
+----------+----------+----------+
```

For n clients, you choose

$$\text{cols} = \lceil \sqrt{n} \rceil$$
$$\text{rows} = \left\lceil \frac{n}{\text{cols}} \right\rceil .$$

Then, for each cell,

$$\text{cell\_w} = \text{screen\_width}/\text{cols}$$
$$\text{cell\_h} = \text{screen\_height}/\text{rows}.$$

A common refinement is to avoid ugly empty cells. For example, with 5 windows:

```
+----------+----------+----------+
|    A     |    B     |    C     |
+----------+----------+----------+
|    D     |          E          |
+----------+---------------------+
```

- **Binary space partitioning**: Binary space partitioning, or BSP, treats the screen as a recursively split rectangle.

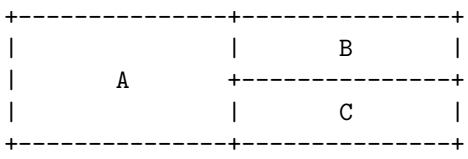
The monitor begins as one rectangle:

```
+------------------------------+
|                              |
|            Root              |
|                              |
+------------------------------+
```

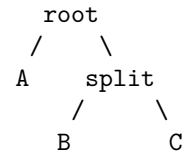
Then you split it

```
+---------------+---------------+
|               |               |
|       A       |       B       |
|               |               |
+---------------+---------------+
```

Then one side can split again:



Conceptually, you get a binary tree:



Each internal node is a split:

```
enum SplitKind {
    SPLIT_HORIZONTAL,
    SPLIT_VERTICAL
};

struct Node {
    bool is_leaf;

    union {
        Client* client;

        struct {
            Node* first;
            Node* second;
            SplitKind split;
            float ratio;
        };
    };
};
```

- **Manual split trees**: Manual split trees are BSP where the user controls where new windows go. For example,
  - Mod+h split horizontally
  - Mod+v split vertically

  Then the next window is inserted according to that chosen split.
- **Dynamic tiling**: Dynamic tiling means the layout is generated automatically from the client list and layout mode.

  In a dynamic tiler, the WM does not store a detailed layout tree for every split. Instead, it stores a list of clients and recomputes positions.

```
clients = [A, B, C, D]
layout = MASTER_STACK
master_factor = 0.6
```

Every time something changes, you call

```
arrange(workspace);
```

Events that trigger rearrangement include:

- new window mapped,
- window closed,
- focus changed,
- layout changed,
- master factor changed,
- monitor resized,
- gap size changed,
- client moved to/from floating state.

Dynamic tiling is simpler than manual tiling.

- **Floating layer**: Not every window should tile.

  Some windows should float:

  - dialog boxes,
  - file pickers,
  - small utility windows,
  - splash windows,
  - transient windows,
  - user-forced floating windows,
  - windows with fixed size hints.

  The important design point is that floating windows are managed by the WM but excluded from tiling.

- **Gaps**: Gaps are empty space between windows

  There are usually two kinds

  - **Outer gaps**: Space between the monitor edge and the window layout
  - **Inner gaps**: Space between adjacent windows

- **Borders**: Border width is part of the windows outer size. If you call

```
XMoveResizeWindow(display, win, x, y, w, h);
```

the w and h refer to the inner window size, not including the border.

- **Resize hints**: Resize hints come from the client application. In X11, you get them with:

```
XGetWMNormalHints
```

They are stored in **XSizeHints**. Important fields include

```
PMinSize      minimum width/height
PMaxSize      maximum width/height
PBaseSize     base width/height
PResizeInc    resize increment
PAspect       aspect ratio constraints
```

Some applications do not want arbitrary sizes.

- **Aspect-ratio constraints**: Aspect-ratio hints tell the WM that a window wants to stay within a certain width/height ratio.

For example,

```
min aspect = 16/10
max aspect = 16/9
```

The aspect ratio is

$$\text{ratio} = \text{width}/\text{height}.$$

If the width is too large for the height, reduce width.

If the height is too large for the width, reduce height.

Layout recomputation

- **Intro**: A tiling window manager should treat the screen layout as a derived result of its internal state.

  That is the main idea behind layout recomputation. You do **not** want the geometry of each window to be the primary thing you manually maintain. Instead, you keep a clean model of the current world, then regenerate window positions from that model whenever something important changes.

  state -> layout algorithm -> window rectangles -> XMoveResizeWindow calls

  The window positions are an output, they are not the real source of truth.

- **Why you recompute the whole layout**: Suppose you have three windows A, B, C, and window B closes. If you only manually remove B, you now need to decide what happens to C.

  The better model is

  - **Before:**: clients = [A, B, C]
  - **After:**: clients = [A, C]

  Then rerun the layout algorithm.

- **New window appears**: When a new window appears, you usually receive something like MapRequest. You should

  1. decide whether to manage the window,
  2. add it to the current workspace's client list,
  3. set focus if appropriate,
  4. recompute the layout,
  5. map the window.

- **Window close**: When a window closes, you remove it from your model. Do not just "erase the dead window from the screen." The X server already destroys/unmaps it. Your job is to update your internal state and then recompute the layout for the remaining windows.

Focus model

- **Intro:** A tiling window manager needs a focus model because X11 does not automatically know which client should receive keyboard input. Your WM must decide which window is "active," tell the X server where keyboard events should go, and keep that decision consistent when windows are created, destroyed, hidden, moved between workspaces, or moved between monitors.

  In X11 terms, focus mostly means keyboard focus. Pointer/mouse events are separate. A window can receive mouse events because the pointer is over it, while a completely different window receives keyboard events because it has input focus.

- **Input focus**: Input focus is the X11 mechanism that determines which window receives keyboard events.

  The core call is usually

  ```
  XSetInputFocus(display, window, RevertToPointerRoot,
  ↪  CurrentTime);
  ```

  The focused window then receives keyboard input, assuming it selected the relevant events or has descendants that do.

  A window manager usually does not focus the frame window for typing. It normally focuses the actual client window.

  You may decorate/reparent the client into a frame, but keyboard focus should usually go to the client, not the decoration frame.

  Focus is not just an X server state. Your WM also needs its own internal state saying which client is focused.

- **Click-to-focus**: Click-to-focus means a window becomes focused only when the user clicks it.

  This is the easiest policy to implement and reason about.

- **Focus-follows-mouse**: Focus-follows-mouse means the window under the pointer becomes focused when the pointer enters it. The usual X event is **EnterNotify.**

  you should usually ignore some EnterNotify events generated by grabs, ungrabs, or window hierarchy changes. X11 crossing events have fields like mode and detail that help distinguish normal pointer motion from synthetic/side-effect transitions.

  You do not want accidental focus changes caused by:

  - Mapping a new window
  - Unmapping a window
  - Moving/resizing a window
  - Grabbing/ungrabbing pointer

  - Switching workspace
  - Restacking windows

  So a real implementation often filters enter events.

- **Sloppy focus**: Variant of FFM. The difference is what happens when the pointer leaves a window and enters empty root-window space.

  - **Pointer over window A**: A focused
  - **Pointer over root**: A remains focused

  Sloppy focus is often better for tiling WMs because there may be gaps, bars, or empty areas. You usually do not want focus to disappear just because the pointer moved over the root window.

- **Focus direction in a tiling layout**: A tiling WM often supports commands like:

  - focus left
  - focus right
  - focus up
  - focus down

  There are two common ways to implement directional focus.

  - **Layout-tree focus**: If your layout is stored as a tree, you navigate the tree
  - **Geometry-based focus**: You compare window rectangles. If focused client is A, and the user says focus right, find the visible client whose rectangle is to the right of A and closest by distance.

Keyboard control

- **Intro**: In a tiling WM, keyboard control is not "text input." It is a global command interface. You grab certain key combinations on the root window, receive KeyPress events before normal clients do, translate those events into internal commands, then mutate your WM state: focus, layout, workspace, client order, spawning, quitting, and so on.

```
physical key press
    -> X11 KeyCode + modifier mask
    -> WM matches key binding
    -> dispatches command
    -> command mutates WM state
    -> WM rearranges / focuses / spawns / exits
```

- **Passive grabs**: A passive key grab means: "when this key combination is pressed, send the event to my WM."

  In Xlib, this is usually done with **XGrabKey.**

  XGrabKey establishes a passive keyboard grab. When the matching key and modifier combination is pressed, the grab becomes active and the key event is delivered to the grabbing client, usually your WM.

  For a WM, you usually grab keys on the root window, because the root window is the parent of all top-level client windows. This gives you "global" WM keybindings.

- **Modifier keys**: Modifiers are bitmask attached to key events

```
ShiftMask
LockMask       // usually Caps Lock
ControlMask
Mod1Mask       // often Alt
Mod2Mask       // often Num Lock
Mod3Mask
Mod4Mask       // often Super / Windows key
Mod5Mask
```

  Most tiling WMs use Mod4Mask as the main WM modifier:

- **Key symbols**: A KeySym is the symbolic meaning of a key. For example,

```
XK_Return
XK_Tab
XK_space
XK_j
XK_k
XK_h
XK_l
XK_1
XK_q
```

These are layout-level symbolic names. You generally write your config using KeySyms because they are readable.

Internally you store keysyms, then when registering a grab you convert it to a keycode.

- **Key codes**: A KeyCode is the physical-ish code X11 reports for a key on the keyboard.

  Your config should usually use KeySym.

  Your grabs require KeyCode

- **Spawn commands**: A WM does not usually implement a terminal, launcher, browser, or editor itself. It spawns external programs.

  A spawn command usually forks, closes the X connection in the child, starts a new session, then execs

Mouse control

- **Intro**: A pure tiling window manager can avoid most mouse logic, but a practical one usually still needs mouse handling for:
  - moving floating windows;
  - resizing floating windows;
  - interacting with title bars, borders, or decorations;
  - allowing Mod + mouse controls;
  - clicking the root window for menus or launchers;
  - dragging windows between monitors or workspaces.

  In Xlib, mouse control is mostly built out of button events, motion events, and pointer grabs.
- **Button grabs**: A button grab tells the X server: "When this mouse button is pressed with this modifier combination, send the event to the window manager."

  This is usually how WMs implement things like:
  - Mod + Left Mouse = move floating window
  - Mod + Right Mouse = resize floating window

  We use **XGrabButton**.
- **Passive grabs**: A passive grab is created ahead of time with **XGrabButton**. It says "Wake me up when this mouse button/modifier combination happens."
- **Active grabs**: An active pointer grab is created with XGrabPointer.

  It says: "From now until I ungrab, send pointer motion/release events to me."

  This is useful during dragging or resizing. Once a move operation starts, you do not want to lose motion events if the pointer leaves the window.
- **Dragging windows**: Dragging a floating window is mostly geometry math.

  You usually want to move the frame window, not the raw client window, if your WM reparents clients.

  If your WM is non-reparenting, then you move the client window directly.
- **Resizing floating windows**: Resizing is similar to moving, but instead of changing x and y, you usually change width and height.
- **Root-relative coordinates**: Root-relative coordinates are coordinates relative to the root window.

  These are usually the best coordinates for dragging and resizing because they are stable globally.
- **Client-relative coordinates**: Client-relative coordinates are relative to the event window.

Borders and visual feedback

- **Intro**: In an X11 tiling window manager, borders and visual feedback are how the WM tells the user:
  - "this window is focused,"
  - "this window is urgent,"
  - "this window is fullscreen,"
  - "this window belongs to the current layout."

  Without good visual feedback, a tiling WM feels disorienting even if the layout algorithm works correctly.

- **Borders**: In X11, a top-level application window can have a border controlled through the X server.

  The classic calls are

  ```
  XSetWindowBorderWidth(display, window, width);
  XSetWindowBorder(display, window, pixel_color);
  ```

  The border belongs to the X window itself. Your WM does not need to draw a rectangle manually around the client unless you use reparenting.

  For a simple non-reparenting tiling WM, changing the client window's border is usually enough.

- **Focused border color**: The focused window should be visually obvious. The focus border is one of the most important parts of the WM. In a tiling environment, the mouse may not clearly indicate which window receives keyboard input, so the border must.

- **Unfocused border color**: Every managed but unfocused tiled window usually gets a neutral border.

- **Border width**: Border width affects both appearance and geometry.

  If you set

  ```
  XSetWindowBorderWidth(dpy, win, 2);
  ```

  Then, the total visible rectangle is

  ```
  outer_width  = client_width  + 2 * border_width
  outer_height = client_height + 2 * border_width
  ```

- **Urgent window color**: Urgency is usually used when a background window wants attention. For example:

  - A terminal bell occurs.
  - A chat window receives a message.
  - A dialog appears on another workspace.
  - A program sets the urgency hint.

  This is commonly exposed through **WM_HINTS**. You can query hints with **XGetWMHints**. You usually clear urgency when the user focuses the window.

  Urgency is not the same as focus. An urgent window can be unfocused but still have a special border.

- **Fullscreen border suppression**: Fullscreen windows usually should not have borders. When a window is fullscreen, users expect it to occupy the entire monitor:
- **Note:** Do not make focus, layout, and border code all fight each other. A clean model is
  - focus logic decides which client is focused
  - layout logic decides where clients go
  - state logic decides fullscreen/urgent/floating
  - border logic visually reflects that state

  So instead of directly setting borders from everywhere, have one function that says:

```
redraw_client_decoration(client);
```

  Or

```
update_client_decoration(client);
```

  That function should be the only place where focused, unfocused, urgent, and fullscreen border decisions are made.

Workspaces

- **Workspace**: A workspace is a logical collection of windows. Only one workspace might be visible on a given monitor at a time.

  In a simple WM, each window belongs to exactly one workspace.

  So when you switch from workspace 1 to workspace 2, you do something like:

```
hide_workspace(old_workspace);
show_workspace(new_workspace);
arrange_workspace(new_workspace);
```

  Note that hide usually means unmapping, and show usually means mapping.

- **Tags**: A tag is similar to a workspace, but more flexible.
  - **Workspace model**: A window belongs to one workspace
  - **Tag model**: A window may have one or more tags

  For example, in a tag-based WM like dwm, a window can be tagged as both dev and web.
  - **Terminal:**: tags = {1}
  - **Browser:**: tags = {2}
  - **Docs:**: tags = {1, 2}

  If you view tag one you see *Terminal* and *Docs*. If you view tag two you see *Browser* and *Docs*

- **Multi-monitor**: You need to decide whether workspaces are
  1. Global, like i3-style workspaces that move between monitors.
  2. Per-monitor, where each monitor has its own independent workspace set.
  3. Tag-based, where each monitor views a tag mask.
- **Per-workspace layout**: Most tiling WMs allow each workspace to remember its own layout. So, a workspace might store

```
enum LayoutKind {
    LAYOUT_TILE,
    LAYOUT_MONOCLE,
    LAYOUT_FLOATING,
    LAYOUT_GRID
};

struct Workspace {
    Client* clients;
    Client* focused;
    enum LayoutKind layout;
};
```

When you switch workspaces, you do not globally switch the whole WM's layout. You use the layout stored inside the workspace:

- **Sticky windows**: A stick window appears on all workspaces. So when switching workspaces, sticky windows remain mapped:
- **Workspaces and EWMH**: If you want your WM to cooperate with panels, pagers, taskbars, and desktop utilities, you should expose workspace state through EWMH properties.

Important root window properties include:

```
_NET_NUMBER_OF_DESKTOPS
_NET_CURRENT_DESKTOP
_NET_DESKTOP_NAMES
_NET_CLIENT_LIST
_NET_CLIENT_LIST_STACKING
```

Important client properties include:

```
_NET_WM_DESKTOP
_NET_WM_STATE
_NET_WM_STATE_STICKY
```

For example,

```
_NET_WM_DESKTOP = 0
```

Means the window is on desktop / workspace 0. Sticky windows are commonly represented with:

```
_NET_WM_DESKTOP = 0xFFFFFFFF
```

or with

```
_NET_WM_STATE_STICKY
```

Configure requests

- **Intro**: In Xorg, configure requests are how a client says: "I would like my top-level window to be moved, resized, raised, lowered, or otherwise reconfigured."

  In a normal X session without a window manager, the X server would just do it. With a window manager running, the WM intercepts those requests and decides what actually happens.

  This is central to a tiling WM because tiling is based on the rule: The layout owns the geometry, not the application.

  So when a tiled terminal asks to resize itself to 900x600, your WM usually says, "No, your tile is 960x540; stay there."

- **What a configure request is**: A client can call functions such as:

```
XMoveWindow(display, win, x, y);
XResizeWindow(display, win, width, height);
XMoveResizeWindow(display, win, x, y, width, height);
XConfigureWindow(display, win, mask, &changes);
```

  For a top-level application window, this does not immediately happen if your WM has selected SubstructureRedirectMask on the root window.

  Instead, the X server sends your WM a **ConfigureRequest** event. The event type is

```
XConfigureRequestEvent
```

  If a client is tiled, you should ignore the configure request. For floating windows, dialogs, popups, and special utility windows, you usually do honor the request.

  A good WM does not blindly honor every request. It still checks:

  - minimum size
  - maximum size
  - resize increments
  - aspect ratio
  - screen bounds
  - struts/panels

- **Struts and panels**: In the Linux graphical environment, panels (or taskbars) house application launchers and system trays, while struts are the invisible "margins" that tell the window manager to keep applications from covering those panels. They work together to reserve screen space for your desktop UI.

- **Synthetic ConfigureNotify events**: This is one of the most important practical details.

Applications expect to receive **ConfigureNotify** events when their geometry changes. But if your WM intercepts a configure request and decides not to change the window, the application may still need to be told: "Here is your actual geometry."

This is done with a **synthetic ConfigureNotify event**. The WM creates the event manually and sends it to the client. Do this when the client requested geometry, the WM denied or modified the request, and the client still needs to know its real position/size.

- **Window gravity**: Window gravity controls how a window's position should be interpreted when its size changes. The basic question is: If the window changes size, which point should stay fixed?

  Common gravities include:

```
0 NorthWestGravity
1 NorthGravity
2 NorthEastGravity
3 WestGravity
4 CenterGravity
5 EastGravity
6 SouthWestGravity
7 SouthGravity
8 SouthEastGravity
9 StaticGravity
```

Mapping, unmapping, and destruction

- **Intro**: In a tiling X11 window manager, this section is about separating X11 window state from your WM's internal client state.

  A Window ID can exist, be mapped, be unmapped, be destroyed, be assigned to a workspace, be hidden, or be known to your WM. These are related, but they are not the same thing.

- **Two layers**: You should think in two layers

  1. **X server state**:
     - Does this X window exist?
     - Is it mapped or unmapped?
     - Has it been destroyed?
  2. **WM state**:
     - Do I have a Client struct for it?
     - Which workspace is it on?
     - Is it visible in the current layout?
     - Is it focused?
     - Is it floating / tiled / fullscreen?

  A common beginner mistake is treating "unmapped" as "dead." That is wrong.

  An unmapped window may be:

  - a client that exited,
  - a client that voluntarily withdrew itself,
  - a window hidden because it is on another workspace,
  - an iconified/minimized window,
  - a window your WM temporarily unmapped while rearranging.

  The X server generates **MapNotify** and **UnmapNotify** when a window changes between mapped and unmapped states. DestroyNotify is generated when a client destroys a window with XDestroyWindow or related operations.

- **Managing a window**: To manage a window means: your WM creates internal state for it. When you manage a window, you usually: When you manage a window, you usually:

  1. allocate a Client,
  2. store the Window ID,
  3. select events on the client window,
  4. insert it into your client list,
  5. assign it to the current workspace,
  6. tile it,
  7. map it if it should be visible,
  8. possibly focus it.

- **Mapping**: For a tiling WM, mapping usually happens when

- a new client appears,
- you switch to the workspace containing that client,
- a hidden/minimized client is restored,
- a client changes state from withdrawn/iconic to normal.

- **Unmapping**: Means to ask the X server to make it not visible. An UnmapNotify can mean very different things depending on who caused it.

  (a) **The client unmapped itself**: The application may be withdrawing its top-level window. In ICCCM terms, newly created top-level windows begin in the Withdrawn state, and transitions between Withdrawn, Normal, and Iconic are driven by mapping, unmapping, and certain WM messages.

  If a real client window unmaps itself, your WM will often treat that as:
  - client is no longer normal/manageable
  - remove it from tiling data
  - clear focus if needed

  That is an **unmanage** operation.

  (b) **Your WM unmapped it to hide it**: Suppose the user switches from workspace 1 to workspace 2.

```
for each client on old_workspace:
    XUnmapWindow(dpy, client->win);

for each client on new_workspace:
    XMapWindow(dpy, client->win);
```

  But those **XUnmapWindow** calls will also produce **UnmapNotify.**

  You must not accidentally interpret those as "the client exited." The client is still managed. It is merely hidden.

```
client->hidden = true;
client->ignore_unmap++;
XUnmapWindow(dpy, client->win);
```

  Then in UnmapNotify:

```
void handle_unmap_notify(XUnmapEvent *e) {
    Client *c = find_client(e->window);
    if (!c) return;

    if (c->ignore_unmap > 0) {
        c->ignore_unmap--;
        return;
    }

    unmanage(c);
}
```

The important point is that you should not blindly unmanage every window that receives **UnmapNotify.**

- **Unmanaging a window**: To unmanage a window means to delete your WM's internal record for it. This usually means

```
void unmanage(Client *c) {
    if (!c) return;

    if (focused == c)
        focused = NULL;

    remove_from_workspace(c);
    remove_from_client_list(c);

    free(c);

    arrange();
    focus_next_valid_client();
}
```

Unmanaging should happen when the client is truly gone from the WM's perspective. Reasons include:

  - the client destroyed its top-level window,
  - the client withdrew itself,
  - the WM decided this window should no longer be managed,
  - startup cleanup found stale clients.

Unmanage must be idempotent in spirit. In other words, your code should be robust if multiple events arrive close together.

Note: Idempotent describes a property of certain actions or operations that yield the same exact result, regardless of whether they are executed once or multiple times. If you repeat the action, it causes no further change beyond the initial application

For example, a client may unmap and then destroy. If your **UnmapNotify** handler already unmanaged it, your **DestroyNotify** handler must not free the same Client again.

- **Destroyed windows**: A destroyed window is different from an unmapped window. Destroyed means

  - The X server object is gone.
  - The Window ID is no longer valid for normal use.

After **DestroyNotify,** you should assume the Window ID is dead. Do not try to focus it, configure it, move it, resize it, restack it, or read properties from it unless you are deliberately handling X errors.

Destruction cleanup should clear:

focused client, last focused client, urgent client pointer, fullscreen client pointer, workspace client references, layout stack references, any pointer stored in monitor/screen state.

- **Withdrawn state**: In ICCCM terminology, a top-level client window can be in states such as:

  1. **Withdrawn**: Baseline state of a top-level window. It means the window is unmapped, invisible, and completely ignored by the window manager.
  2. **Normal**: Application's top-level window should be fully visible, active, and viewable on the screen.
  3. **Iconic**: Minimized

  A newly created top-level window starts in the Withdrawn state. It leaves Withdrawn when the client maps it and the WM begins managing it.

  When the client maps, the WM handles **MapRequest**, creates a client, and tiles it.

  When the client withdraws itself, the WM receives an unmap-related event and should remove the client from its managed list. ICCCM specifically notes that WMs should handle the transition to Withdrawn on real UnmapNotify for compatibility.

- **Client exits**: When an application exits, its windows are usually destroyed by the X server. The WM may see

  – UnmapNotify

  – DestroyNotify

  or just one of them depending on timing and event selection. Your logic should tolerate both. A safe model is

  – **UnmapNotify**: If caused by WM hiding/minimizing, ignore as lifecycle removal. Otherwise unmanage.

  – **DestroyNotify**: If still managed, unmanage. If not managed, ignore.

Stacking order

- **Intro**: In X11, windows are not only arranged in 2D space with x, y, width, and height. They also have a Z-order, called the stacking order.

  The stacking order determines which window appears in front of another when they overlap.

  Even in a tiling window manager, where normal windows usually do not overlap, stacking still matters because of:

  - floating windows
  - fullscreen windows
  - dialogs
  - popups
  - panels/docks
  - menus
  - transient windows
  - focused-window visual behavior

  A tiling window manager still needs a clear stacking policy.

- **Raising**: To raise a window means to move it higher in the stacking order.

  ```
  0  XRaiseWindow(display, window);
  ```

  This places the window above its siblings. In a tiling WM, you usually do not need to raise every focused tiled window, because tiled windows normally do not overlap. But you may still raise focused floating windows.

- **Lowering**: To lower a window means to move it lower in the stacking order.

  ```
  0  XLowerWindow(display, window);
  ```

  Lowering is less common in tiling WMs, but it can be useful when implementing:

  - "send to back"
  - demoting floating windows
  - keeping tiled windows below floating windows
  - restoring a normal window after fullscreen mode

- **Restack**: To restack means to explicitly reorder multiple windows.

  ```
  0  XRestackWindows(display, windows, count);
  ```

  A tiling WM often benefits from having a function like:

```
void restack_workspace(Workspace* ws);
```

That function can enforce your intended stacking layers every time layout changes.

- **Stacking Policy**: A common stacking policy is

```
top
    fullscreen
    docks/panels
    floating
    tiled
bottom
```

Fullscreen should also interact with panels/docks. Some WMs allow fullscreen windows to cover panels. Others keep panels visible. That is a policy decision.

So, another valid policy is

```
top
    docks/panels
    fullscreen
    floating
    tiled
bottom
```

- **Transient windows above parents**: A transient window is a temporary child-like window associated with another top-level window.

  Examples:

  – dialog boxes
  – file picker windows
  – confirmation prompts
  – settings popups
  – "Save changes?" windows

  In X11, a client can mark a window as transient using:

```
XGetTransientForHint(display, win, &parent);
```

  If a window is transient for another window, your WM should usually keep it above its parent.

- **Focused window stacking**: Some WMs raise the focused window. Others do not.

In a floating WM, focusing a window usually raises it. For tiled windows, raising on focus usually does not matter because they should not overlap.

- **ConfigureRequest and stacking**: In X11, applications may request changes to their own stacking order. The WM receives these as **ConfigureRequest** events if it selected **SubstructureRedirectMask** on the root window.

  A client might request:

  - raise me
  - lower me
  - place me above another window
  - place me below another window
  - change my size
  - change my position

  A WM is not required to blindly obey these requests. The WM owns global window policy.

  For example, if a tiled client requests to raise itself above a fullscreen window, you probably reject or reinterpret that request.

- **Sibling windows**: Stacking order is among sibling windows. In X11, top-level application windows are usually children of the root window. Since they share the same parent, they are siblings.

Error handling

- **Intro**: In Xlib, error handling is different from normal C error handling because many X requests do not fail immediately. You often send a request to the X server, continue executing, and only later receive an error event saying that an earlier request was invalid.
- **Errors are asynchronous**: When you call something like

```
XMoveResizeWindow(...)
```

  That function usually does not wait for the X server to confirm success. It writes a request into Xlib's output buffer and returns.

  The actual error, if any, may occur later when the server processes the request.

- **The X error handler**: Xlib lets you install a global error handler.

```
int x_error_handler(Display* display, XErrorEvent* error) {
    char msg[1024];

    XGetErrorText(display, error->error_code, msg,
    ↪  sizeof(msg));

    fprintf(stderr,
        "X error: %s\n"
        "  request_code: %d\n"
        "  minor_code: %d\n"
        "  resourceid: 0x%lx\n",
        msg,
        error->request_code,
        error->minor_code,
        error->resourceid
    );

    return 0;
}
```

  Then, install it

```
XSetErrorHandler(x_error_handler);
```

  The handler is called when the X server reports an error for one of your requests.

  Important: the error handler is **global per process**, not per request. You cannot easily attach a normal callback to one X request.

- **Bad window**: BadWindow means you tried to use a window ID that the X server no longer considers valid.

This is extremely common in window managers.

For example, in

```
XMoveWindow(display, win, 100, 100);
```

A **BadWindow** might occur if the client destroyed the window before your request reached the server. A tiling WM should treat many **BadWindow** errors as normal race conditions, not fatal bugs.

But do not blindly ignore everything. A **BadWindow** can also indicate that your WM state is stale or corrupted.

- **BadAccess**: BadAccess means your request attempted to access something exclusively controlled by another client.

  For a window manager, the most important case is this:

  ```
  XSelectInput(display, root, SubstructureRedirectMask |
  ↪  SubstructureNotifyMask);
  ```

  Only one client can select **SubstructureRedirectMask** on the root window. If another WM is already running, your call will generate **BadAccess.**

- **Detecting another WM already running**: Because errors are asynchronous, this is not enough:

  ```
  XSelectInput(display, root, SubstructureRedirectMask);
  printf("I am the WM now\n");
  ```

  The error may not have arrived yet. You need to force the server to process the request using **XSync**.

```c
static int wm_detected = 0;

int wm_error_handler(Display* display, XErrorEvent* error) {
    if (error->error_code == BadAccess) {
        wm_detected = 1;
        return 0;
    }

    return 0;
}

int main() {
    Display* display = XOpenDisplay(NULL);
    if (!display) {
        fprintf(stderr, "Could not open display\n");
        return 1;
    }

    XSetErrorHandler(wm_error_handler);

    Window root = DefaultRootWindow(display);

    XSelectInput(display, root,
        SubstructureRedirectMask |
        SubstructureNotifyMask
    );

    XSync(display, False);

    if (wm_detected) {
        fprintf(stderr, "Another window manager is already
        ↪  running\n");
        return 1;
    }

    printf("Window manager started\n");

    // Main event loop...
}
```

The key line is

```c
XSync(display, False);
```

That forces pending requests to be sent to the server and waits until the server has processed them. The second argument controls whether pending events are discarded. Do not use True casually because it discards pending events.

- **Good handler**: A good error handler should print at least

```c
int x_error_handler(Display* display, XErrorEvent* e) {
    char buffer[1024];

    XGetErrorText(display, e->error_code, buffer,
    ↪  sizeof(buffer));

    fprintf(stderr, "X error: %s\n", buffer);
    fprintf(stderr, "  error_code: %d\n", e->error_code);
    fprintf(stderr, "  request_code: %d\n", e->request_code);
    fprintf(stderr, "  minor_code: %d\n", e->minor_code);
    fprintf(stderr, "  resourceid: 0x%lx\n", e->resourceid);
    fprintf(stderr, "  serial: %lu\n", e->serial);

    return 0;
}
```

Field	Meaning
error_code	The actual error, such as BadWindow or BadAccess.
request_code	The major X protocol request that failed.
minor_code	Extra request-specific code, especially useful for extensions.
resourceid	The X resource involved, often a Window, Pixmap, GC, etc.
serial	The request serial number associated with the error.

The serial is useful because Xlib assigns sequence numbers to requests. Advanced debugging can compare the error serial to request serials.

- **Common errors**:
  - **BadWindow**: You tried to operate on a destroyed or invalid window.
  - **BadAccess**: You requested access that another client already owns.
  - **BadMatch**: The resources are valid, but incompatible for that request.
  - **BadDrawable**: You passed an invalid Drawable.
  - **BadValue**: One of your numeric arguments was invalid.
  - **BadAlloc**: Server could not allocate resource.

Startup sequence / minimal implementation order

- **Startup sequence**: A basic WM startup usually does this:
  1. Open display.
  2. Get root window.
  3. Install X error handler.
  4. Select events on root window.
  5. Grab keys/buttons.
  6. Scan existing windows.
  7. Manage existing windows.
  8. Enter event loop.
- **Minimal implementation order**: A good build order is
  1. Start the WM and claim the root window
  2. Handle MapRequest
  3. Track managed clients
  4. Tile all windows in a simple vertical stack
  5. Add focus
  6. Add keybindings
  7. Add master-stack layout
  8. Add window closing
  9. Add workspaces
  10. Add floating windows
  11. Add fullscreen
  12. Add ICCCM/EWMH support
  13. Add multi-monitor suppor

ICCCM / EWMH Discussion

- **Window managers**: An X11 window manager is just another X client. It is not privileged. It becomes "the window manager" by selecting SubstructureRedirectMask on the root window. Once it does that, map/configure/restack requests for top-level client windows are redirected to it as events such as MapRequest, ConfigureRequest, and CirculateRequest. Only one client can select SubstructureRedirectMask on a given window, which is why only one WM can manage a root window at a time.

  ICCCM is the base contract. It defines things like WM_NORMAL_HINTS, WM_HINTS, WM_PROTOCOLS, WM_DELETE_WINDOW, WM_TAKE_FOCUS, WM_TRANSIENT_FOR, and WM_STATE. EWMH explicitly builds on ICCCM and says compliant WMs/clients must adhere to ICCCM where applicable

  For a tiling WM, ICCCM keeps normal programs from breaking. EWMH makes your WM cooperate with panels, launchers, taskbars, pagers, fullscreen applications, and modern toolkits.

- **ICCCM state model**: ICCCM says a top-level client is viewed by the WM as being in one of three states:

  - **NormalState**: The client window is viewable
  - **IconicState**: The client is iconified/minimized
  - **WithdrawnState**: The client is not managed or visible

  New top-level windows start in the withdrawn state. The WM places a WM_STATE property on each managed top-level window that is not withdrawn. This is done with **XChangeProperty**.

  When you manage a window, set WM_STATE to NormalState. When you unmanage it because it withdrew or was destroyed, remove or update WM_STATE.

- **WM_NORMAL_HINTS (geometry constraints)**: Clients use WM_NORMAL_HINTS to describe preferred size behavior: minimum size, maximum size, base size, resize increments, aspect ratio, and window gravity. ICCCM says the WM interprets client configure requests similarly to initial geometry and that clients must be prepared for the WM not to allocate exactly what they requested.

  Important fields are

```
PMinSize      // min_width, min_height
PMaxSize      // max_width, max_height
PBaseSize     // base_width, base_height
PResizeInc    // width_inc, height_inc
PAspect       // min_aspect, max_aspect
PWinGravity   // win_gravity
USPosition    // user-specified position
USSize        // user-specified size
PPosition     // program-specified position
PSize         // program-specified size
```

For tiling, the big one is PResizeInc. Terminals often want sizes in character-cell increments. If you ignore increments, terminals still work, but you may get ugly unused padding. A simple first WM can ignore this; a better WM applies increments after computing the tile rectangle.

- **WM_HINTS**: WM_HINTS carries extra information from the client to the WM. Important fields include:

  Important fields are

```
InputHint
StateHint
IconPixmapHint
IconWindowHint
WindowGroupHint
UrgencyHint
```

  The input field participates in ICCCM focus handling. UrgencyHint means the client wants attention; the ICCCM says the WM must make some effort to draw the user's attention while this flag is set.

- **Focus models and WM_TAKE_FOCUS**: ICCCM defines four input models:

  ICCCM defines four input models:

Model	WM_HINTS.input	WM_TAKE_FOCUS
No Input	False	absent
Passive	True	absent
Locally Active	True	present
Globally Active	False	present

  Passive and locally active clients set input = True, meaning the WM may set focus to their top-level window. No-input and globally active clients set input = False, meaning the WM should not directly set focus to that top-level window. Clients with WM_TAKE_FOCUS may receive a ClientMessage from the WM containing WM_TAKE_FOCUS and a valid timestamp.

- **More on focus**: The window manager does not know how every application internally handles focus.

  Some applications are simple. They have one top-level window, and keyboard input can go directly to that top-level window.

  Other applications have many child windows: text fields, editor panes, toolbars, embedded widgets, terminal grids, browser views, and so on. In those cases, the top-level window may not be the real input target. The application may want focus to go to one of its own subwindows instead

That is the reason ICCCM defines focus models. The client tells the WM:

1. Whether the WM should directly set focus to the client's top-level window.
2. Whether the client wants a WM_TAKE_FOCUS message so it can choose what to do internally

These two pieces of information come from WM_HINTS.input and WM_PROTOCOLS containing WM_TAKE_FOCUS

ICCCM defines four input models using those two values.

1. **Model 1: No Input**:

   ```
   WM_HINTS.input = False
   WM_TAKE_FOCUS absent
   ```

   This client never wants keyboard input. The WM should not call XSetInputFocus, the WM also does not send WM_TAKE_FOCUS
2. **Model 2: Passive Input**:

   ```
   WM_HINTS.input = True
   WM_TAKE_FOCUS absent
   ```

   This is the simplest normal application model. The client wants keyboard input, but it does not manage focus itself. It expects the WM to set focus to the client's top-level window.

   In this case we can call XSetInputFocus, but no WM_TAKE_FOCUS message is needed.
3. **Model 3: Locally Active Input**:

   ```
   WM_HINTS.input = True
   WM_TAKE_FOCUS present
   ```

   This client wants WM assistance, but it also participates in focus management.

   The WM may set focus to the top-level window because input = True. The WM may also send WM_TAKE_FOCUS because the client advertised that protocol.

   This model is for clients that can manage focus among their own subwindows, but only after they already have focus or after the WM offers focus.

   ```
   XSetInputFocus(dpy, c->win, RevertToPointerRoot, time);
   send_wm_take_focus(c, time);
   ```

   Then the client may respond by setting focus to one of its own child windows.
4. **Model 4: Globally Active Input**:

```
WM_HINTS.input = False
WM_TAKE_FOCUS present
```

This is the most subtle model. The client wants keyboard input, but it does not want the WM to blindly set focus to its top-level window.

Instead, the WM should send WM_TAKE_FOCUS. The client then decides whether to accept focus and which internal window should receive it.

ICCCM says No Input and Globally Active clients set input = False, which requests that the WM not set input focus to their top-level window

- **What is WM_TAKE_FOCUS**: $\mathrm{WM}_{T}AK\mathrm{E}_{F}OCUSisnotafunctioncall.Itisanatomusedinsidea ClientMes$

```
WM_PROTOCOLS = [..., WM_TAKE_FOCUS, ...]
```

The WM reads WM_PROTOCOLS. If WM_TAKE_FOCUS is present, the WM can send the client a message saying: "You may take focus now, using this timestamp."

- **Note about timestamps**: The time argument matters. X uses timestamps to prevent stale clients from stealing focus after a newer user action. ICCCM specifically says clients setting focus should use the timestamp of the event that caused the focus attempt, not CurrentTime. FocusIn cannot be used for this because FocusIn events do not carry timestamps
- **WM_PROTOCOLS: polite close and focus protocol**: The two protocols you really care about are:

```
WM_DELETE_WINDOW
WM_TAKE_FOCUS
```

When the user asks to close a window, do not blindly call XKillClient or XDestroyWindow. First check whether the window supports WM_DELETE_WINDOW.

ICCCM says clients that include WM_DELETE_WINDOW in WM_PROTOCOLS receive a ClientMessage with data[0] = WM_DELETE_WINDOW, and WMs should not use DestroyWindow on a window that has this protocol.

- **ConfigureRequest: tiled versus floating policy**: When a tiled client asks to resize itself, your WM does not have to obey. In tiling mode, your layout owns geometry.

  But ICCCM expects the client to be told what its actual geometry is. If the WM denies a configure request, it should send a synthetic ConfigureNotify describing the unchanged geometry. If it changes the geometry, it may also need to send a synthetic ConfigureNotify, especially for reparenting WMs.

- **Transients and dialogs**: WM_TRANSIENT_FOR tells you that a window is transient for another top-level window. This usually means dialog, modal popup, file chooser, alert, etc.

In a tiling WM, transient windows should usually be floating, centered over their parent, and kept above the parent. Do not tile every dialog into the master stack unless you intentionally want that behavior.

EWMH also defines _NET_WM_WINDOW_TYPE; _NET_WM_WINDOW_TYPE_DIALOG, _NET_WM_WINDOW_TYPE_SPLASH, _NET_WM_WINDOW_TYPE_UTILITY, and _NET_WM_WINDOW_TYPE_DOCK are especially relevant. The spec says _NET_WM_WINDOW_TYPE should be used by the WM to determine decoration, stacking, and behavior; if a managed window lacks _NET_WM_WINDOW_TYPE but has WM_TRANSIENT_FOR, it must be treated as a dialog.

- **EWMH properties you should implement early**: You can write a usable WM with only ICCCM, but modern programs behave better if you implement a small EWMH subset.

  The root-window properties to implement are

  1. **_NET_SUPPORTED**: List atoms your WM supports
  2. **_NET_SUPPORTING_WM_CHECK**: Identifies your WM
  3. **_NET_CLIENT_LIST**: Managed windows in mapping order
  4. **_NET_CLIENT_LIST_STACKING**: Managed windows in stacking order
  5. **_NET_NUMBER_OF_DESKTOPS**: Number of workspaces
  6. **_NET_CURRENT_DESKTOP**: Current workspace
  7. **_NET_ACTIVE_WINDOW**: Currently focused window
  8. **_NET_WORKAREA**: Usable screen area after docks/panels

  The active window property is read-only from the client's perspective; clients request activation by sending a _NET_ACTIVE_WINDOW client message to the root, and the WM may accept or refuse it.

  Client-window properties/messages to support:

  1. **_NET_WM_NAME**: UTF-8 title
  2. **_NET_WM_WINDOW_TYPE**: normal/dialog/dock/utility/etc.
  3. **_NET_WM_STATE**: fullscreen, sticky, above, demands attention
  4. **_NET_WM_DESKTOP**: workspace assignment
  5. **_NET_WM_PID**: process ID, useful for diagnostics
  6. **_NET_ACTIVE_WINDOW**: focus request
  7. **_NET_CLOSE_WINDOW**: polite close request
  8. **_NET_WM_STATE_FULLSCREEN:** fullscreen behavior

  For a tiling WM, _NET_WM_STATE_FULLSCREEN is the most important modern state. A fullscreen window should usually cover the monitor work area or full monitor area, be above tiled windows, and suppress normal tiling while active.

- **Fullscreen**: When a client requests _NET_WM_STATE_FULLSCREEN:

  1. Mark client fullscreen.
  2. Save old geometry.
  3. Move/resize it to monitor bounds.
  4. Raise it.

5. Focus it.
6. Update _NET_WM_STATE

When fullscreen is removed:

- Restore old geometry if floating.
- Otherwise return it to tiling layout.
- Re-run arrange()

- **WM_COLORMAP_WINDOWS**: This is mostly legacy, but ICCCM says the WM is responsible for installing/uninstalling colormaps for managed top-level clients. Clients give hints through their top-level colormap attribute or WM_COLORMAP_WINDOWS.

  On modern TrueColor desktops, this usually does not matter much. But technically, an ICCCM-aware WM should watch colormap properties/attributes and install the appropriate colormap when a client gets colormap focus.

- **Panels, docks, and struts**: If you want compatibility with bars/panels, support:

```
_NET_WM_WINDOW_TYPE_DOCK
_NET_WM_STRUT
_NET_WM_STRUT_PARTIAL
_NET_WORKAREA
```

  A dock/panel should usually be floating and not tiled. Its strut reserves screen space. Your tiling area should exclude that reserved space.

- **Protocols**: In ICCCM, a window manager protocol is a named convention between a client window and the window manager.

  The client advertises which protocols it understands by setting the $\mathrm{WM}_{P}ROTOCOLSpropertyonitstop-levelwindow$.

```
WM_PROTOCOLS = [
    WM_DELETE_WINDOW,
    WM_TAKE_FOCUS,
    _NET_WM_PING
]
```

  The client is saying "Window manager, I know how to participate in these specific conversations."

  The ICCCM defines WM_PROTOCOLS as a property of type ATOM; each atom in the list identifies a communication protocol between the client and WM.

  Without protocols, a WM would have to do blunt things. For example, closing a window could mean:

```
XKillClient(dpy, window);
```

or

```
XDestroyWindow(dpy, window);
```

But that is rude. The application may need to ask the user to save a file, cancel a download, close a database connection, or reject the close.

So ICCCM defines WM_DELETE_WINDOW. If the client lists WM_DELETE_WINDOW in WM_PROTOCOLS, the WM should send a message saying: "The user requested that you close." Then, the client decides what to do.

So, a protocol is a structured interaction.

- **Common WM protocols**:
  - **WM_DELETE_WINDOW**: Used for polite window closing. The client receives it and may close itself, ask the user to save, ignore it, or do something else.

    ICCCM says WMs should not use DestroyWindow on a window that has WM_DELETE_WINDOW in WM_PROTOCOLS.
  - **WM_TAKE_FOCUS**: Used for focus negotiation. The WM sends a `ClientMessage` saying "You may take keyboard focus now, using this timestamp."

    The client may then call XSetInputFocus on itself or one of its child windows.

    This is used by clients that need control over their internal focus behavior.
  - **_NET_WM_PING**: This is EWMH, not old ICCCM, but it also uses $WM_PROTOCOLS$.

  A client can list _NET_WM_PING in WM_PROTOCOLS. The WM can use it to check whether the client is still processing events, especially after a close request seems to hang.

- **ClientMessage**: A ClientMessage is an X event type used for client-to-client communication. Xlib documentation says the X server generates ClientMessage events only when a client calls XSendEvent.

```
Client A calls XSendEvent
        ↓
X server delivers synthetic event
        ↓
Client B receives ClientMessage
```

  Client A is often the WM, client B is an application window.

- **XClientMessageEvent**:

```
typedef struct {
    int type;               // ClientMessage
    unsigned long serial;
    Bool send_event;
    Display *display;
    Window window;
    Atom message_type;
    int format;
    union {
        char b[20];
        short s[10];
        long l[5];
    } data;
} XClientMessageEvent;
```

Set `type` to `ClientMessage`, and `window` to the window the message is about to be delivered to. For WM_DELETE_WINDOW, this is the client window being asked to close.

`message_type` is the general category of the message. For ICCCM WM protocols, this is usually

```
WM_PROTOCOLS
```

For EWMH requests, this may be something like:

```
_NET_ACTIVE_WINDOW
_NET_WM_STATE
_NET_CLOSE_WINDOW
```

`format` is the size of the data elements. `Data` is the payload. For ICCCM WM_PROTOCOLS, the first value usually identifies the specific protocol. For example,

```
data.l[0] = WM_DELETE_WINDOW;
data.l[1] = timestamp;
```

- **Sending WM_DELETE_WINDOW**: First, intern the atoms

```
Atom WM_PROTOCOLS;
Atom WM_DELETE_WINDOW;

WM_PROTOCOLS = XInternAtom(dpy, "WM_PROTOCOLS", False);
WM_DELETE_WINDOW = XInternAtom(dpy, "WM_DELETE_WINDOW",
    ↪  False);
```

Then, check whether the client supports it.

```
Bool supports_protocol(Display* dpy, Window w, Atom
↪  protocol) {
    Atom* protocols = NULL;
    int count = 0;

    if (!XGetWMProtocols(dpy, w, &protocols, &count))
    return False;

    Bool found = False;

    for (int i = 0; i < count; ++i) {
        if (protocols[i] == protocol) {
            found = True;
            break;
        }
    }

    XFree(protocols);
    return found;
}
```

Then,

```
Bool supports_protocol(Display* dpy, Window w, Atom
↪  protocol) {
    Atom* protocols = NULL;
    int count = 0;

    if (!XGetWMProtocols(dpy, w, &protocols, &count))
    return False;

    Bool found = False;

    for (int i = 0; i < count; ++i) {
        if (protocols[i] == protocol) {
            found = True;
            break;
        }
    }

    XFree(protocols);
    return found;
}
```

For strict ICCCM behavior, prefer a real event timestamp instead of **CurrentTime** when you have one.

The WM sends the message, and the client decides how to react.

- **Receiving ClientMessage in the WM**: The WM also receives ClientMessage events.

Modern clients often send EWMH requests to the root window. For example:

```
_NET_ACTIVE_WINDOW
_NET_WM_STATE
_NET_CLOSE_WINDOW
```

A client might send _NET_ACTIVE_WINDOW to ask: “Please activate/focus this window.”

For ICCCM WM_PROTOCOLS messages, you normally send directly to the client that owns the destination window:

```
XSendEvent(dpy, w, False, NoEventMask, &ev);
```

NoEventMask has a special meaning here: send it to the client that created the destination window.

For EWMH messages sent to the root window, clients usually use masks such as:

```
SubstructureRedirectMask | SubstructureNotifyMask
```

because the WM selected those masks on the root.

- **send_event**: Every X event structure has a send_event field through the common event header. It is set to true if the event came from a SendEvent protocol request.